

# DOCKER\_TASK\_3

Create a image from running container.

Copy image from local machine to docker server and load the image.

Create Docker image using alpine and customize with tomcat.

1. Create single stage and multi stage docker file using this source code:  
<https://github.com/betawins/multi-stage-example.git>
2. Install docker compose and execute sample application.
3. Implement solution to scan images when pushed to docker registry.
4. Implement solution to scan images when pushed to AWS ECR.
5. Create a Jenkins pipeline to create a docker image and push the image to Docker hub

Done bro ☒ we'll do ONLY these tasks now, step-by-step with exact commands + files.  
(You can copy paste directly in your EC2/Linux machine)

## TASK 1: Create an Image from Running Container

### Step 1: Run a container

```
→docker container run -itd -p 80:80 --name nginx nginx:latest
```

```
root@ip-172-31-27-214:~# docker container run -itd -p 80:80 --name nginx nginx:latest
Unable to find image 'nginx:latest' locally
latest: Pulling from library/nginx
0c8d55a45c0d: Pull complete
46bf3a120c8e: Pull complete
4f4efe02d542: Pull complete
7b6cb8ccac7b: Pull complete
673400e223fd: Pull complete
47cd406a84ef: Pull complete
bae5a1799a80: Pull complete
Digest: sha256:341bf0f3ce6c5277d6002cf6e1fb0319fa4252add24ab6a0e262e0056d313208
Status: Downloaded newer image for nginx:latest
a464622d4034ec9246c9bc855cfdc365e8f87475ad4d1fcfc4636b91c4865e0f
root@ip-172-31-27-214:~#
```

i-028f1dc9ea91e8b89 (Docker\_controller)

PublicIPs: 54.221.2.185 PrivateIPs: 172.31.27.214

### Step 2: Modify container content

```
→docker exec -it nginx /bin/sh
```

```
→echo "<h1>Hello from running container commit</h1>" >
/usr/share/nginx/html/index.html
```

```
→exit
```

```

root@ip-172-31-27-214:~# docker ps
CONTAINER ID   IMAGE     COMMAND                  CREATED     STATUS      PORTS                               NAMES
a464622d4034   nginx:latest "/docker-entrypoint..." 2 minutes ago Up 2 minutes 0.0.0.0:80->80/tcp, [::]:80->80/tcp   nginx
root@ip-172-31-27-214:~# docker exec -it nginx /bin/sh
# echo "Hello from container" > /usr/share/nginx/html/index.html
# exit
root@ip-172-31-27-214:~#

```

### Step 3: Create image from running container

```
docker commit nginx mynginx-commit:v1
```

### Step 4: Verify

```
docker images
```

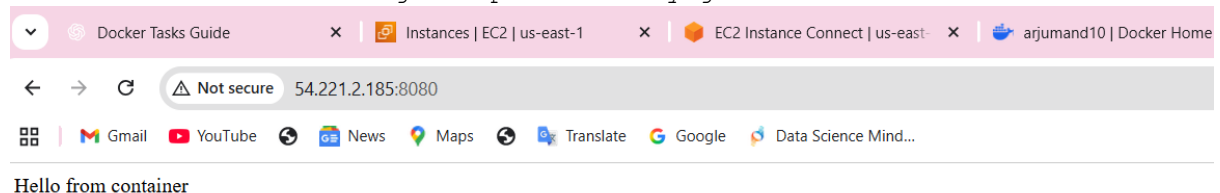
```

root@ip-172-31-27-214:~# docker ps
CONTAINER ID   IMAGE     COMMAND                  CREATED     STATUS      PORTS                               NAMES
a464622d4034   nginx:latest "/docker-entrypoint..." 2 minutes ago Up 2 minutes 0.0.0.0:80->80/tcp, [::]:80->80/tcp   nginx
root@ip-172-31-27-214:~# docker exec -it nginx /bin/sh
# echo "Hello from container" > /usr/share/nginx/html/index.html
# exit
root@ip-172-31-27-214:~# docker commit nginx mynginximage:v1
sha256:bf43af6d7194c6971e6f88f1c46a11580e026df6ace7651daa4f5f569b6f12b4
root@ip-172-31-27-214:~# docker images
REPOSITORY    TAG       IMAGE ID       CREATED        SIZE
mynginximage   v1        bf43af6d7194   21 seconds ago 161MB
nginx          latest    5cdef4ac3335   35 hours ago   161MB
root@ip-172-31-27-214:~#

```

### Step 5: Run new container using committed image

```
docker run -itd --name nginx -p 8080:80 mynginx-commit:v1
```



Access:

👉 <http://<public-ip>:8080>

## TASK 2: Copy Image from Local Machine to Docker Server and Load

### Step 1: Save image into tar

```
→ docker save -o nginx.tar nginx:latest
```

```

ajumand@ajumand:~/Downloads (master)
$ docker pull nginx
Using default tag: latest
latest: Pulling from library/nginx
ae5a1799a80: Pulling fs layer
e8d55a45c0d: Pulling fs layer
6bf3a120c8e: Pulling fs layer
b6cb8ccac7b: Pulling fs layer
73400a233fd: Pulling fs layer
f4efe02d542: Pulling fs layer
7cd406a84ef: Pulling fs layer
73400a233fd: Download complete
b6cb8ccac7b: Download complete
ae5a1799a80: Download complete
f4efe02d542: Download complete
7cd406a84ef: Download complete
e8d55a45c0d: Download complete
6bf3a120c8e: Download complete
e8d55a45c0d: Pull complete
f4efe02d542: Pull complete
6bf3a120c8e: Pull complete
73400a233fd: Pull complete
b6cb8ccac7b: Pull complete
ae5a1799a80: Pull complete
7cd406a84ef: Pull complete
Digest: sha256:341bf0f3ceb6c5277d6002cf6e1fb0319fa4252add24ab6ae26e0056d313208
Status: Downloaded newer image for nginx:latest
docker.io/library/nginx:latest

ajumand@ajumand:~/Downloads (master)
$ docker images
REPOSITORY   TAG       IMAGE ID       CREATED        SIZE
nginx        latest    341bf0f3ceb6   35 hours ago   237MB

ajumand@ajumand:~/Downloads (master)
$ docker save -o mynginximage.tar mynginximage:v1
error response from daemon: no such image: mynginximage:v1

ajumand@ajumand:~/Downloads (master)
$ docker save -o nginx.tar nginximage:v1
error response from daemon: No such image: nginximage:v1

ajumand@ajumand:~/Downloads (master)
$ docker save -o nginx.tar nginx:v1
error response from daemon: no such image: nginx:v1

```

## Step 2: Copy tar file to another server (Docker server)

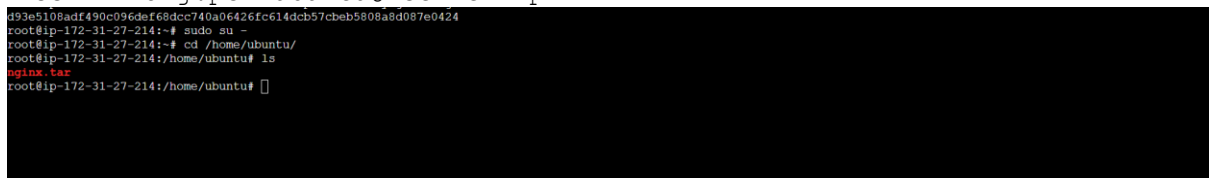
From your local machine:

→ `scp -i arj.pem nginx.tar ubuntu@<server-ip>:/home/ubuntu/`



## Step 3: Login to server

→ `ssh -i arj.pem ubuntu@<server-ip>`



## Step 4: Load image

→ `docker load -i /home/ubuntu/nginx.tar`



## Step 5: Verify

→ `docker images`

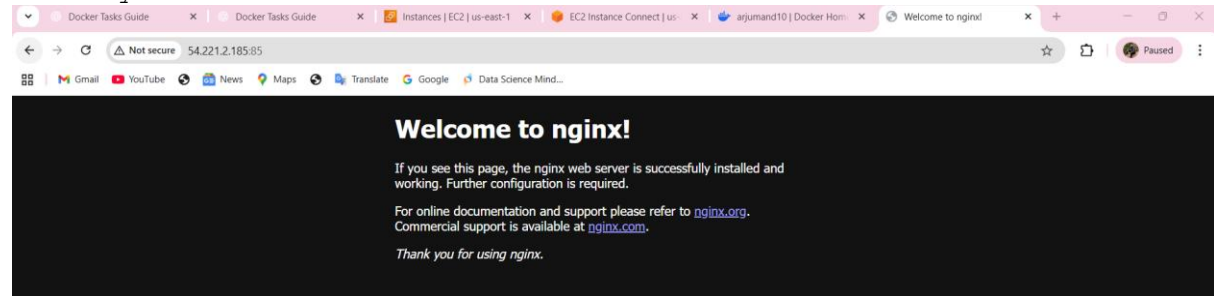


## Step 6: Run container

→ `docker run -itd -p 9090:80 nginx:v1`

```
root@ip-172-31-27-214:~# docker load -i /home/ubuntu/nginx.tar
Loaded image: nginx:latest
root@ip-172-31-27-214:~# docker ps
CONTAINER ID   IMAGE      COMMAND                  CREATED        STATUS        PORTS                               NAMES
8464622d4034   nginx:latest "/docker-entrypoint..." About an hour ago Up About an hour 0.0.0.0:80->80/tcp, [::]:80->80/tcp  nginx
root@ip-172-31-27-214:~# docker run -itd -p 85:80 nginx:latest
31f0c2a73972e97e97733cbd905532e35193ef55a6d3476104a90922f41eb5ee
root@ip-172-31-27-214:~#
```

→ verify on web



## TASK 3: Create Docker Image using Alpine and Customize with Tomcat

### Create folder

→ `mkdir alpine-tomcat`  
→ `cd alpine-tomcat`

```
root@ip-172-31-27-214:~# mkdir alpine-tomcat
root@ip-172-31-27-214:~# cd alpine-tomcat/
root@ip-172-31-27-214:~/alpine-tomcat#
```

### Create Dockerfile

→ `vi Dockerfile`

Paste this:

```
FROM alpine:latest

RUN apk update && apk add openjdk11 wget tar bash

WORKDIR /opt

RUN wget https://downloads.apache.org/tomcat/tomcat-9/v9.0.96/bin/apache-
tomcat-9.0.96.tar.gz && \
    tar -xvzf apache-tomcat-9.0.96.tar.gz && \
    mv apache-tomcat-9.0.96 tomcat && \
    rm -rf apache-tomcat-9.0.96.tar.gz

EXPOSE 8080
```

```
CMD ["/opt/tomcat/bin/catalina.sh", "run"]
```

```
FROM alpine:latest
RUN apk update && apk add openjdk11 wget tar bash
WORKDIR /opt
RUN wget https://dlcdn.apache.org/tomcat/tomcat-9/v9.0.115/bin/apache-tomcat-9.0.115.tar.gz \
    && tar -xvzf apache-tomcat-9.0.115.tar.gz \
    && mv apache-tomcat-9.0.115 tomcat \
    && rm -rf apache-tomcat-9.0.115.tar.gz
EXPOSE 8080
CMD ["/opt/tomcat/bin/catalina.sh", "run"]
```

## Build image

```
docker build -t alpine-tomcat:v1 .
```

## Run container

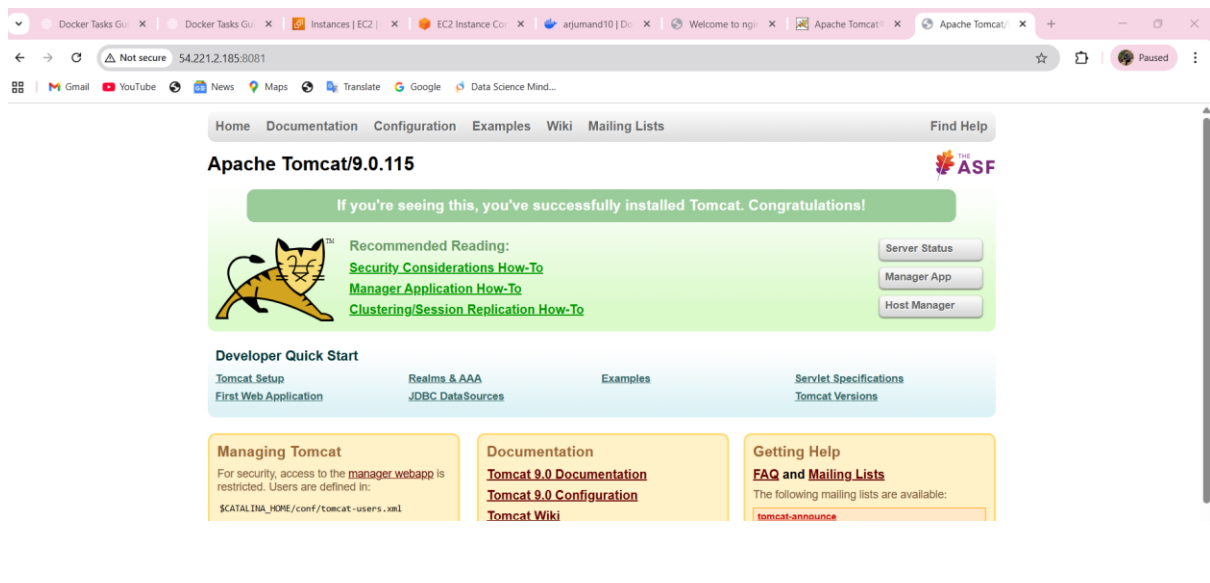
```
docker run -itd --name tomcat-alpine -p 8081:8080 alpine-tomcat:v1
```

```
Successfully built ace9dab20d4d
Successfully tagged alpine-tomcat:v1
root@ip-172-31-27-214:~/alpine-tomcat# docker ps
CONTAINER ID   IMAGE          COMMAND                  CREATED        STATUS        PORTS                               NAMES
31f0c2a73972   nginx:latest   "/docker-entrypoint..." 8 minutes ago  Up 8 minutes  0.0.0.0:85->80/tcp, [::]:85->80/tcp  jovial_pike
a464622d4034   nginx:latest   "/docker-entrypoint..." About an hour ago  Up About an hour  0.0.0.0:80->80/tcp, [::]:80->80/tcp  jovial_pike
4e4e6013810ab9f06eccc3646616848707eb80388cd4b518ed08ed017a564b1 tomcat-alpine -p 8081:8080 alpine-tomcat:v1
root@ip-172-31-27-214:~/alpine-tomcat#
```

i-028f1dc9ea91e8b89 (Docker\_controller)  
PublicIPs: 54.221.2.185 PrivateIPs: 172.31.27.214

Access:

👉 <http://<public-ip>:8081>



## TASK 4: Single Stage and Multi Stage Dockerfile (Git Repo)

Repo:

<https://github.com/betawins/multi-stage-example.git>

## Clone repo

```
git clone https://github.com/betawins/multi-stage-example.git
cd multi-stage-example
```

```
root@ip-172-31-27-214:~# git clone https://github.com/betawins/multi-stage-example.git
Cloning into 'multi-stage-example'...
remote: Enumerating objects: 31, done.
remote: Counting objects: 100% (7/7), done.
remote: Compressing objects: 100% (6/6), done.
remote: Total 31 (delta 2), reused 1 (delta 1), pack-reused 24 (from 1)
Receiving objects: 100% (31/31), 53.25 KiB | 13.31 MiB/s, done.
Resolving deltas: 100% (3/3), done.
root@ip-172-31-27-214:~# cd multi-stage-example/
root@ip-172-31-27-214:~/multi-stage-example#
```

### ◇ 4.1 Single Stage Dockerfile

→ Create Dockerfile.single:

→ vi Dockerfile

→ Paste:

```
FROM maven:3.9.6-eclipse-temurin-17

WORKDIR /app

COPY . .

RUN mvn clean package -DskipTests

EXPOSE 8080

CMD ["java", "-jar", "target/*.jar"]
```

```
FROM maven:3.9.6-eclipse-temurin-17
WORKDIR /app
COPY . .
RUN mvn clean package
EXPOSE 8080
CMD ["java", "-jar", "target/*.jar"]
```

→ Build:

→ docker build -f Dockerfile -t single-stage-app:v1 .

→ Run:

docker run -itd --name singleapp -p 8082:8080 single-stage-app:v1

```
Downloaded from central: https://repo.maven.apache.org/maven2/org/apache/maven/shared/maven-dependency-tree/3.0.1/maven-dependency-tree-3.0.1.jar (37 kB at 167 kB/s)
Downloaded from central: https://repo.maven.apache.org/maven2/org/jdom/jdom2/2.0.6/jdom2-2.0.6.jar (305 kB at 1.3 MB/s)
Downloaded from central: https://repo.maven.apache.org/maven2/org/ow2/asm/asm-util/7.2-beta/asm-util-7.2-beta.jar (81 kB at 348 kB/s)
Downloaded from central: https://repo.maven.apache.org/maven2/org/vafer/jdependency/2.3.0/jdependency-2.3.0.jar (169 kB at 721 kB/s)
Downloaded from central: https://repo.maven.apache.org/maven2/com/google/guava/guava/19.0/guava-19.0.jar (2.3 MB at 8.2 MB/s)
[INFO] Replacing main artifact with repackaged archive
[INFO] -----
[INFO] BUILD SUCCESS
[INFO] -----
[INFO] Total time: 19.708 s
[INFO] Finished at: 2026-02-06T11:48:40Z
[INFO] -----
--> Removed intermediate container 25dba3f1292e
--> ee7208ae3ea0
Step 5/6 : EXPOSE 8080
--> Running in 32a4af376f3f
--> Removed intermediate container 32a4af376f3f
--> 83056dcd30c1
Step 6/6 : CMD ["java", "-jar", "target/*.jar"]
--> Running in 27703e00ed93
--> Removed intermediate container 27703e00ed93
--> 3a15db1cad73
Successfully built 3a15db1cad73
Successfully tagged single-stage-app:v1
root@ip-172-31-27-214:~/multi-stage-example# docker run -itd --name singleapp -p 8082:8080 single-stage-app:v1
9b0395748c2fa3b7f524a389d351a24ad27a02c9131fffa36e876e3a01bfcfd
root@ip-172-31-27-214:~/multi-stage-example#
```

i-028f1dc9ea91e8b89 (Docker\_controller)

PublicIPs: 54.221.2.185 PrivateIPs: 172.31.27.214

→ We can see the size of the single docker file

```
See 'snap info <snapname>' for additional versions.
root@ip-172-31-27-214:~/multi-stage-example# docker images
REPOSITORY          TAG                 IMAGE ID            CREATED             SIZE
single-stage-app     v1                 3a15db1cad73       3 minutes ago      554MB
alpine-tomcat        v1                 ace9dab20d4d       17 minutes ago     300MB
mynginximage         v1                 bf43af6d7194       About an hour ago   161MB
nginx                latest             5cdef4ac3335       36 hours ago       161MB
alpine               latest             a40c03cbb81c       9 days ago         8.44MB
maven                 3.9.6-eclipse-temurin-17 b86805e902bf       2 years ago        481MB
root@ip-172-31-27-214:~/multi-stage-example#
```

i-028f1dc9ea91e8b89 (Docker\_controller)

PublicIPs: 54.221.2.185 PrivateIPs: 172.31.27.214

## 4.2 Multi Stage Dockerfile

Create Dockerfile.multi:

vim Dockerfile.multi

Paste:

```
# Stage 1: Build
FROM maven:3.9.6-eclipse-temurin-17 AS builder

WORKDIR /app
COPY . .
RUN mvn clean package -DskipTests

# Stage 2: Runtime
FROM eclipse-temurin:17-jdk-alpine

WORKDIR /app
COPY --from=builder /app/target/*.jar app.jar

EXPOSE 8080
CMD ["java", "-jar", "app.jar"]
```

```
# Stage 1: Build stage
FROM maven:3.9.6-eclipse-temurin-17 AS builder

WORKDIR /app
COPY . .
RUN mvn clean package -DskipTests

# Stage 2: Runtime stage
FROM eclipse-temurin:17-jdk-alpine

WORKDIR /app
COPY --from=builder /app/target/*.jar app.jar

EXPOSE 8080
CMD ["java", "-jar", "app.jar"]
```

Build:

→docker build -f Dockerfile -t multi-stage-app:v1 .

```
5a547da4fe15: Download complete
2c540078c28: Verifying Checksum
2c540078c28: Download complete
5a547da4fe15: Pull complete
2c540078c28: Pull complete
7cd2433ef004e: Pull complete
946c51efe0fd: Pull complete
Digest: sha256:903e865ecd125a01759fd156d042b998aa86957262f6dec4745125a47bd4f882
Status: Downloaded newer image for eclipse-temurin:17-jdk-alpine
--> d3af761e11a8
Step 6/9 : WORKDIR /app
--> Running in f5fabac3e516
--> Removed intermediate container f5fabac3e516
--> 3b4c97408ee9
Step 7/9 : COPY --from=builder /app/target/*.jar app.jar
--> 7f9ca7d920a3
Step 8/9 : EXPOSE 8080
--> Running in abfa6b5f35c6
--> Removed intermediate container abfa6b5f35c6
--> c3ad1f428c6f
Step 9/9 : CMD ["java", "-jar", "app.jar"]
--> Running in ec6938100a4c
--> Removed intermediate container ec6938100a4c
--> 678b9aca0f4d
Successfully built 678b9aca0f4d
Successfully tagged multi-stage-app:v1
root@ip-172-31-27-214:~/multi-stage-example#
```

Run:

→docker run -itd --name multiapp -p 8083:8080 multi-stage-app:v1

```
Successfully built 678b9aca0f4d
Successfully tagged multi-stage-app:v1
root@ip-172-31-27-214:~/multi-stage-example# docker run -itd --name multiapp -p 8083:8080 multi-stage-app:v1
8df84b3e2320e183195115a534f00539382ba0dcda62f9f3a56634e827519bd0
root@ip-172-31-27-214:~/multi-stage-example#
```

i-028f1dc9ea91e8b89 (Docker\_controller)

PublicIPs: 54.221.2.185 PrivateIPs: 172.31.27.214

## TASK 5: Install Docker Compose and Execute Sample Application

### Install Docker Compose (Ubuntu)

→sudo apt update

→sudo apt install docker-compose -y

```
root@ip-172-31-27-214:~# sudo apt update
sudo apt install docker-compose -y
Hit:1 http://us-east-1.ec2.archive.ubuntu.com/ubuntu noble InRelease
Get:2 http://us-east-1.ec2.archive.ubuntu.com/ubuntu noble-updates InRelease [126 kB]
Get:3 http://us-east-1.ec2.archive.ubuntu.com/ubuntu noble-backports InRelease [126 kB]
Get:4 http://security.ubuntu.com/ubuntu noble-security InRelease [126 kB]
Get:5 http://us-east-1.ec2.archive.ubuntu.com/ubuntu noble-updates/main amd64 Packages [1738 kB]
Get:6 http://us-east-1.ec2.archive.ubuntu.com/ubuntu noble-updates/main Translation-en [324 kB]
Get:7 http://us-east-1.ec2.archive.ubuntu.com/ubuntu noble-updates/main amd64 Components [175 kB]
Get:8 http://us-east-1.ec2.archive.ubuntu.com/ubuntu noble-updates/main amd64 c-n-f Metadata [16.5 kB]
Get:9 http://us-east-1.ec2.archive.ubuntu.com/ubuntu noble-updates/universe amd64 Packages [1528 kB]
Get:10 http://us-east-1.ec2.archive.ubuntu.com/ubuntu noble-updates/universe amd64 Components [386 kB]
Get:11 http://us-east-1.ec2.archive.ubuntu.com/ubuntu noble-updates/universe amd64 c-n-f Metadata [31.9 kB]
Get:12 http://us-east-1.ec2.archive.ubuntu.com/ubuntu noble-updates/restricted amd64 Packages [2582 kB]
```

Verify:

→docker-compose version

```
No containers need to be restarted.

No user sessions are running outdated binaries.

No VM guests are running outdated hypervisor (qemu) binaries on this host.
root@ip-172-31-27-214:~# docker-compose version
docker-compose version 1.29.2, build unknown
docker-py version: 5.0.3
cPython version: 3.12.3
OpenSSL version: OpenSSL 3.0.13 30 Jan 2024
root@ip-172-31-27-214:~#
```

i-028f1dc9ea91e8b89 (Docker\_controller)

PublicIPs: 54.221.2.185 PrivateIPs: 172.31.27.214



## Create sample docker-compose.yml

```
→mkdir compose-demo
→cd compose-demo
→vi docker-compose.yml
```

```
OpenSSL version: OpenSSL 3.0.13 30 Jan 2024
root@ip-172-31-27-214:~# mkdir compose-demo
root@ip-172-31-27-214:~# cd compose-demo/
root@ip-172-31-27-214:~/compose-demo# vi docker-compose.yml
```

i-028f1dc9ea91e8b89 (Docker\_controller)

PublicIPs: 54.221.2.185 PrivateIPs: 172.31.27.214

Paste:

```
version: "3.8"
```

```
services:
  web:
    image: nginx:latest
    container_name: nginx_compose_demo
    ports:
      - "8085:80"
```

```
version: "3.8"
services:
  web:
    image: nginx:latest
    container_name: nginx_compose_demo
    ports:
      - "8085:80"
Run:
docker-compose up -d
Check:
docker ps
```

Run:

```
docker-compose up -d
```

Check:

```
docker ps
```

```
root@ip-172-31-27-214:~/compose-demo# vi docker-compose.yml
vi: command not found
root@ip-172-31-27-214:~/compose-demo# vi docker-compose.yml
root@ip-172-31-27-214:~/compose-demo# docker-compose up -d
Creating network "compose-demo default" with the default driver
Creating nginx_compose_demo ... done
root@ip-172-31-27-214:~/compose-demo# docker ps
```

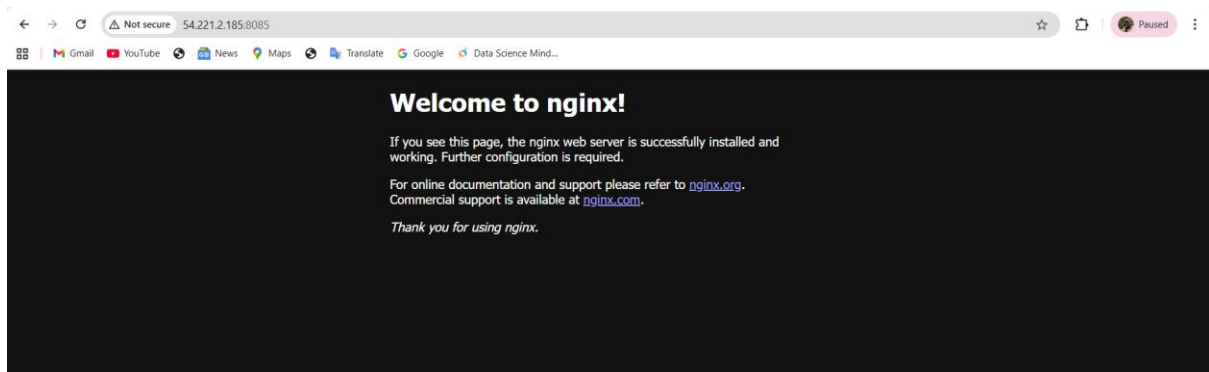
| CONTAINER ID | IMAGE              | COMMAND                  | CREATED           | STATUS           | PORTS                                       | NAMES              |
|--------------|--------------------|--------------------------|-------------------|------------------|---------------------------------------------|--------------------|
| 953a39928b1b | nginx:latest       | "/docker-entrypoint..."  | 13 seconds ago    | Up 12 seconds    | 0.0.0.0:8085->80/tcp, [::]:8085->80/tcp     | nginx_compose_demo |
| 8df84b3e2320 | multi-stage-app:vl | "/_cacert_entrypoint..." | 5 minutes ago     | Up 5 minutes     | 0.0.0.0:8083->8080/tcp, [::]:8083->8080/tcp | multiapp           |
| 4e4e6013810a | alpine-tomcat:vl   | "/opt/tomcat/bin/cat..." | 56 minutes ago    | Up 56 minutes    | 0.0.0.0:8081->8080/tcp, [::]:8081->8080/tcp | tomcat-alpine      |
| 31f0c2a73972 | nginx:latest       | "/docker-entrypoint..."  | About an hour ago | Up About an hour | 0.0.0.0:85->80/tcp, [::]:85->80/tcp         | jovial_pike        |
| 8464622d4034 | nginx:latest       | "/docker-entrypoint..."  | 2 hours ago       | Up 2 hours       | 0.0.0.0:80->80/tcp, [::]:80->80/tcp         | nginx              |

i-028f1dc9ea91e8b89 (Docker\_controller)

PublicIPs: 54.221.2.185 PrivateIPs: 172.31.27.214

Access:

👉 <http://<public-ip>:8085>



Stop:

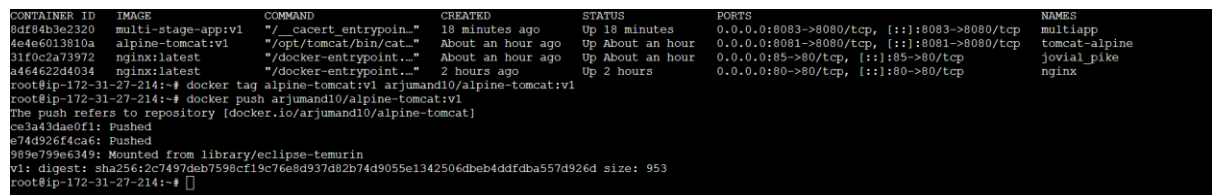
`docker-compose down`



## TASK 6: Scan Images When Pushed to Docker Registry

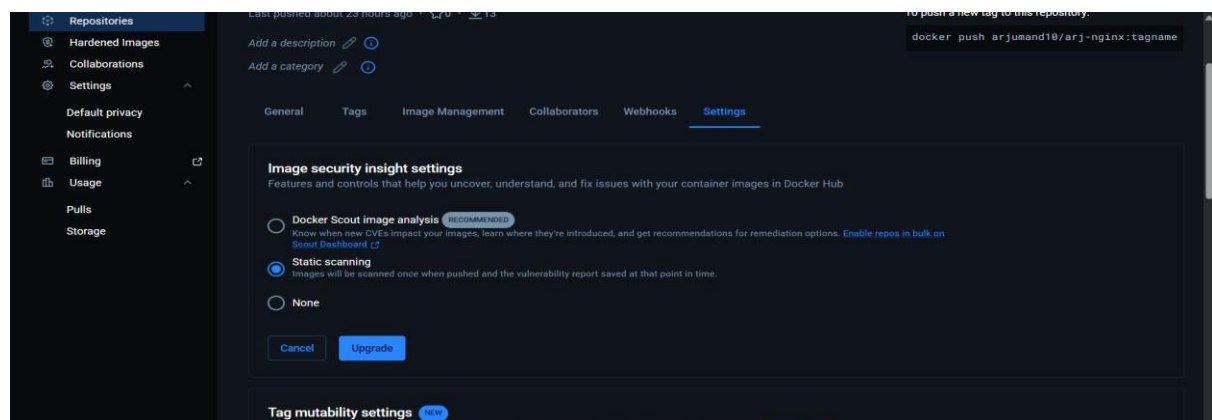
→ `docker tag alpine-tomcat:v1 arjumand10/alpine-tomcat:v1`

→ `docker push arjumand10/alpine-tomcat:v1`

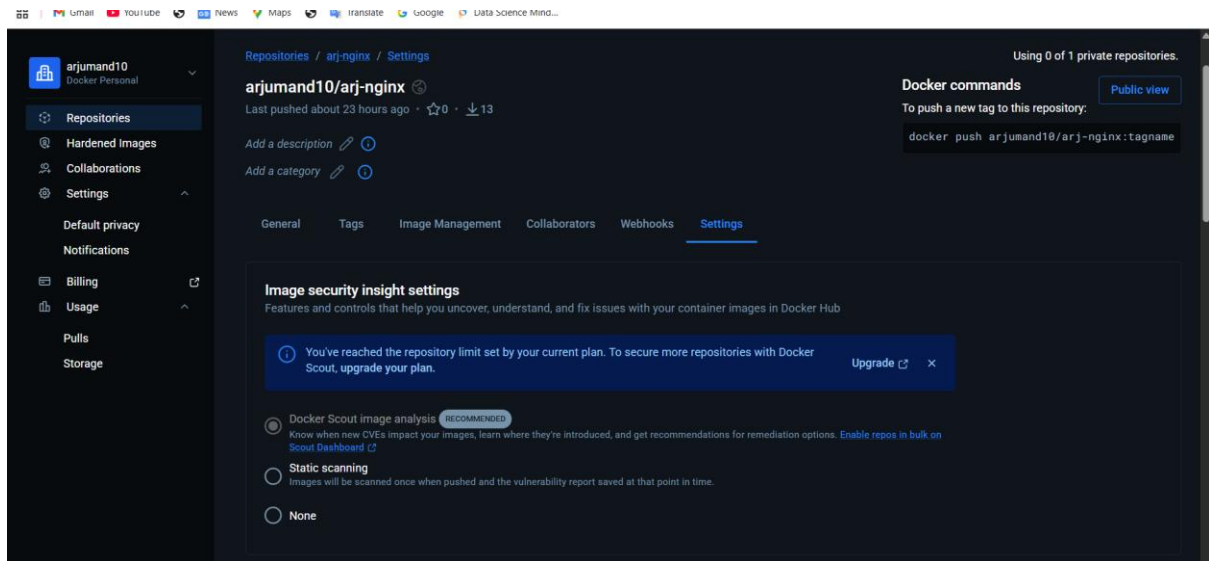


Scan Image before pushing

→ now go to UI

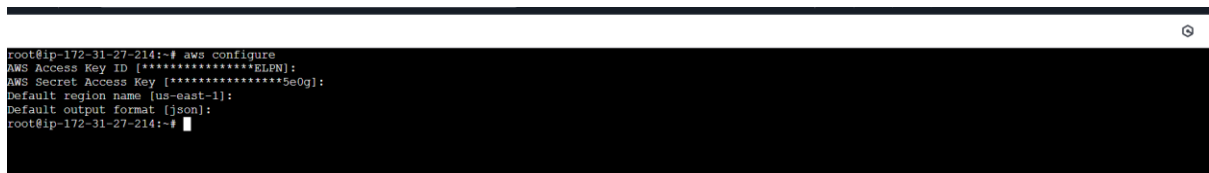


## →enable Docker Scout image analysis



## TASK 7: Scan Images When Pushed to AWS ECR

### →AWS configure

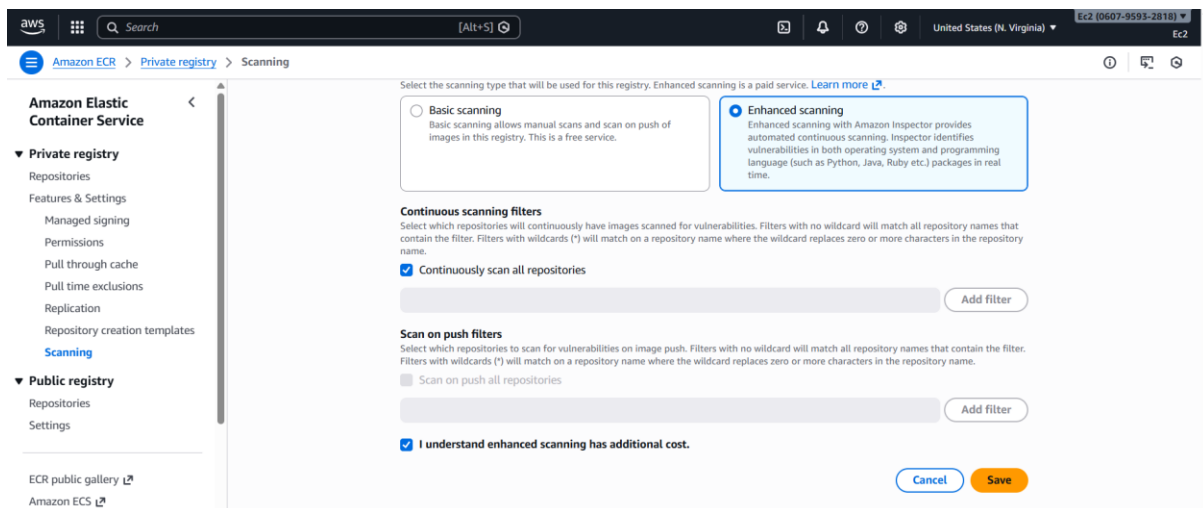


AWS provides scanning using **Amazon Inspector**.

### Enable scanning in ECR

AWS Console → ECR → Repository → Scan settings  
Enable:

☒ Enhanced scanning (Amazon Inspector)



Now when you push images, AWS automatically scans.

→ `aws ecr get-login-password --region us-east-1 | docker login --username AWS --password-stdin 060795932818.dkr.ecr.us-east-1.amazonaws.com`

```
root@ip-172-31-27-214:~# aws ecr get-login-password --region us-east-1 | docker login --username AWS --password-stdin 060795932818.dkr.ecr.us-east-1.amazonaws.com
WARNING! Your credentials are stored unencrypted in '/root/.docker/config.json'.
Configure a credential helper to remove this warning. See
https://docs.docker.com/go/credential-store/
Login Succeeded
```

→ `aws ecr create-repository --repository-name alpine-tomcat --region us-east-1`

```
root@ip-172-31-27-214:~# aws ecr create-repository --repository-name alpine-tomcat --region us-east-1
{
  "repository": {
    "repositoryArn": "arn:aws:ecr:us-east-1:060795932818:repository/alpine-tomcat",
    "registryId": "060795932818",
    "repositoryName": "alpine-tomcat",
    "repositoryUri": "060795932818.dkr.ecr.us-east-1.amazonaws.com/alpine-tomcat",
    "createdAt": "2026-02-06T13:00:27.231000+00:00",
    "imageTagMutability": "MUTABLE",
    "imageScanningConfiguration": {
      "scanOnPush": false
    },
    "encryptionConfiguration": {
      "encryptionType": "AES256"
    }
  }
}

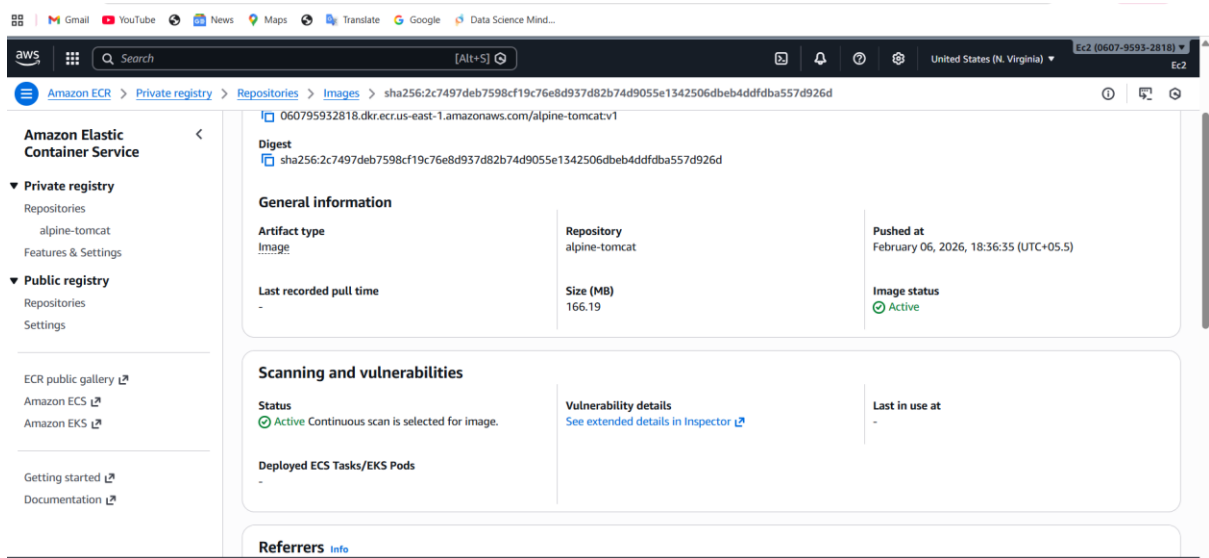
i-028f1dc9ea91e8b89 (Docker_controller)
PublicIPs: 54.221.2.185 PrivateIPs: 172.31.27.214
```

→ `docker tag alpine-tomcat:v1 060795932818.dkr.ecr.us-east-1.amazonaws.com/alpine-tomcat:v1`

→ `docker push 060795932818.dkr.ecr.us-east-1.amazonaws.com/alpine-tomcat:v1`

```
root@ip-172-31-27-214:~# docker tag alpine-tomcat:v1 060795932818.dkr.ecr.us-east-1.amazonaws.com/alpine-tomcat:v1
root@ip-172-31-27-214:~# docker push 060795932818.dkr.ecr.us-east-1.amazonaws.com/alpine-tomcat:v1
The push refers to repository [060795932818.dkr.ecr.us-east-1.amazonaws.com/alpine-tomcat]
e33a43dae0f1: Pushed
e74d926f4ca6: Pushed
989e799e6349: Pushed
v1: digest: sha256:2c7497deb7598cf19c76e8d937d82b74d9055e1342506dbeb4ddfd8a557d92d6 size: 953
root@ip-172-31-27-214:~#
```

Check results:  
ECR → Images → Vulnerability scan findings



## TASK 8: Jenkins Pipeline to Build Docker Image and Push to DockerHub

### Jenkins prerequisites

Install Docker inside Jenkins server:

```
sudo apt update
sudo apt install docker.io -y
sudo usermod -aG docker jenkins
sudo systemctl restart jenkins
```

### Add DockerHub Credentials in Jenkins

Jenkins → Manage Jenkins → Credentials → Global → Add Credentials

→ Open Master Jenkins server and connect with ssh

```
A newer release of "Amazon Linux" is available.
Version 2023.10.20260120:
Version 2023.10.20260202:
Run "/usr/bin/dnf check-release-update" for full release and version update info

#####
Amazon Linux 2023
#####
https://aws.amazon.com/linux/amazon-linux-2023

Last login: Tue Feb  3 13:42:48 2026 from 18.206.107.29
[ec2-user@ip-172-31-6-75 ~]$ sudo su -
Last login: Tue Feb  3 11:36:55 UTC 2026 on pts/1
[root@ip-172-31-6-75 ~]#
```

i-04e9dd922f331d1cf (Master\_jenkin)

PublicIPs: 100.53.218.32 PrivateIPs: 172.31.6.75

- Kind: Username & Password
- ID: dockerhub-creds
- Username: your dockerhub username
- Password: dockerhub password/token

The screenshot shows the Jenkins dashboard with a list of builds. The table below represents the data shown in the image:

| S | W | Name             | Last Success     | Last Failure     | Last Duration |
|---|---|------------------|------------------|------------------|---------------|
| ✓ | 🌥 | Ansible_job      | 2 days 23 hr #13 | 2 days 23 hr #12 | 1 min 37 sec  |
| ✓ | 🌥 | terraform_nodejs | 17 days #3       | 17 days #2       | 4 min 26 sec  |

→ Create a job

The screenshot shows the 'New Item' page in Jenkins. The 'Pipeline' option is highlighted, which is described as: 'Orchestrates long-running activities that can span multiple build agents. Suitable for building pipelines (formerly known as workflows) and/or organizing complex activities that do not easily fit in free-style job type.'

## Jenkinsfile (Build + Scan + Push)

```
pipeline {
    agent any

    environment {
        IMAGE_NAME = "arjumand10/multi-stage-app"
        DOCKERHUB_CREDS = credentials('dockerhub-creds')
    }

    stages {

        stage("Git Clone") {
            steps {
                git "https://github.com/betawins/multi-stage-example.git"
            }
        }

        stage("Build Docker Image") {
            steps {
                sh "docker build -t $IMAGE_NAME:latest ."
            }
        }

        stage("Scan Docker Image") {
            steps {
                sh "trivy image --exit-code 1 --severity HIGH,CRITICAL
$IMAGE_NAME:latest"
            }
        }

        stage("DockerHub Login") {
            steps {
                sh "echo $DOCKERHUB_CREDS_PSW | docker login -u
$DOCKERHUB_CREDS_USR --password-stdin"
            }
        }

        stage("Push Image to DockerHub") {
            steps {
                sh "docker push $IMAGE_NAME:latest"
            }
        }
    }

    post {
        success {
            echo "✅ Docker Image Built, Scanned, and Pushed
Successfully!"
        }
        failure {
            echo "❌ Pipeline Failed. Fix vulnerabilities or errors."
        }
    }
}
```

→verify with IP

**Whitelabel Error Page**

This application has no explicit mapping for /error, so you are seeing this as a fallback.

Fri Feb 06 14:14:53 GMT 2026  
There was an unexpected error (type=Not Found, status=404).  
No message available

---

**Jenkins** Docker Job

Status ✖ Docker Job Add description

Changes  
Build Now  
Configure  
Delete Pipeline  
Full Stage View  
Stages  
Rename  
Pipeline Syntax

**Stage View**

|                                            | Git Clone | Fix Dockerfile (Java17 + SSL Fix) | Build Docker Image | Tag Docker Image | DockerHub Login | Push Image to DockerHub | DockerHub Logout | Remove Old Container | Run Container | Verify | Declarative: Post Actions |
|--------------------------------------------|-----------|-----------------------------------|--------------------|------------------|-----------------|-------------------------|------------------|----------------------|---------------|--------|---------------------------|
| Average stage times: (full run time: ~57s) | 283ms     | 352ms                             | 41s                | 341ms            | 510ms           | 11s                     | 329ms            | 323ms                | 579ms         | 377ms  | 121ms                     |
| #17 Feb 06 2026 No Changes                 | 283ms     | 352ms                             | 41s                | 341ms            | 510ms           | 11s                     | 329ms            | 323ms                | 579ms         | 377ms  | 121ms                     |

**Permalinks**

- Last build (#16), 5 min 43 sec ago
- Last stable build (#10), 45 min ago
- Last successful build (#10), 45 min ago

**Builds**

Filter /

Today  
#17 2:56 PM

## ✓ Final Verification Commands

### Check containers

- docker volume ls
- Check in repo now

**Docker Hub** Explore My Hub

Search Docker Hub CtrlK

Repositories

All repositories within the arjmand10 namespace.

Search by repository name All content Create a repository

| Name                      | Last Pushed       | Contains | Visibility | Scout    |
|---------------------------|-------------------|----------|------------|----------|
| arjmand10/multi-stage-app | 2 minutes ago     | IMAGE    | Public     | Inactive |
| arjmand10/alpine-tomcat   | about 2 hours ago | IMAGE    | Public     | Inactive |
| arjmand10/arj-nginx       | 1 day ago         | IMAGE    | Public     | Active   |

1-3 of 3

## 🌀 OUTPUT EXPECTED AFTER COMPLETING TASKS

- ✓ Image created using docker commit
- ✓ Image transferred using docker save + scp + docker load



- ✓ Alpine Tomcat image created and running
  - ✓ Single-stage & multi-stage images built successfully
  - ✓ Docker-compose installed and app running
  - ✓ Docker registry scan solution using Trivy
  - ✓ AWS ECR scan enabled using Amazon Inspector
  - ✓ Jenkins pipeline builds + scans + pushes image to DockerHub
- 

If you want, I can now write a **full lab document format** (like assignment report) with headings + screenshots section.